

チーム 23 ハッカソン 1
Processing 音源サブシステム
基本設計・詳細設計 仕様書

梅澤颯太（学籍番号：25G1021）

2026 年 4 月 29 日 改訂案

目次

1	本書の位置づけ	3
1.1	前提と参照資料	3
1.2	担当タスクとの対応	3
2	基本設計：機能一覧とシステム構成図	4
2.1	Processing 側 FBS	4
2.2	ユーザー目線のシステム構成	4
2.3	パート対応	5
2.4	性能指標	5
3	基本設計：操作インターフェースと状態遷移	6
3.1	ユーザー操作	6
3.2	画面表示	6
3.3	状態遷移	6
4	詳細設計：ハードウェア設計	8
4.1	構成要素	8
4.2	接続設計	8
5	詳細設計：ソフトウェア設計	9
5.1	ファイル構成	9
5.2	NOTE パケット定義	9
5.3	クラス設計	10
5.4	音色パラメータ	10
5.5	FFT 解析の設計	11
6	詳細設計：処理フローの設計	12
6.1	起動処理	12
6.2	NOTE 受信フロー	12
6.3	ボイス管理フロー	12
6.4	例外処理	12

6.5	主要コード断片	13
7	検証計画	15
7.1	ループリック対応表	15
8	今後の実装順序	15
9	生成 AI の利用について	16

1 本書の位置づけ

本書は、チーム 23「Arduino オーケストラ」における梅澤担当範囲、すなわち Processing 側の音色合成・NOTE 受信・発音管理を対象とした基本設計および詳細設計である。初版のレビューでは、音色方針は具体的である一方、機能一覧、操作インターフェース、状態遷移、NOTE 受信処理、4 パート発音管理の粒度が不足していると指摘された。そこで本改訂案では、システム設計書ループリックの 5 項目すべてに対応するよう、実装に必要な接続・データ・クラス・処理分岐・検証条件までを明文化する。

1.1 前提と参照資料

本書は、以下のチーム資料と整合することを前提とする。

資料	本書で採用する内容
arduino_塩澤.pdf	主構成は金管 3 + ドラム 1, Mac 4 台, 各楽器ノードと各 Mac を USB Serial で 1 対 1 接続する。NOTE は共通ヘッダ 12B + NOTE ペイロード 8B の 20B 固定長バイナリとする。
review_processing_梅澤.md	不足指摘である、機能一覧、状態遷移、バイナリ NOTE 受信、ボイス管理、ドラム音色、ADSR 具体化を本書で補う。
docs/overview.md	プロジェクト目的は、音楽未経験でも指揮によりテンポを変えながら演奏できる体験である。
ADR-0004	指揮者 1 台 + 楽器 4 台, Processing 側で音色合成・再生を行う構成を採用する。
ADR-0006	同期誤差の上位目標は 20 ms 以内とする。Processing 側では NOTE 受信後の発音遅延を 5 ms 以下に抑え、Arduino 側の同期設計を劣化させない。
processing_test.pde	Processing の Serial 受信実験が存在するため、本書ではこれを発展させて固定長バイナリ NOTE の受信処理に置き換える。

1.2 担当タスクとの対応

WBS	タスク	本書での設計箇所
117	音色合成 基本設計	第 2 章の機能一覧、第 5.4 節の金管・ドラム音色方針
124	Arduino から届く NOTE を受信して鳴らす仕組み	第 5.2 節、第 6.2 節
125	4 種類の楽器パートを鳴らす管理	第 5.3 節、第 6.3 節
126	金管楽器風音色の具体実装	第 5.4 節、第 6.4 節
127	FFT 解析による音色根拠づけ	第 5.5 節、第 7 章

2 基本設計：機能一覧とシステム構成図

2.1 Processing 側 FBS

Processing 音源サブシステムを機能分解構造（FBS）として表 3 に示す．塩澤設計の F4「発音情報配信」に対して，本書では PC 側の受信境界から音声出力までを F6 として定義する．

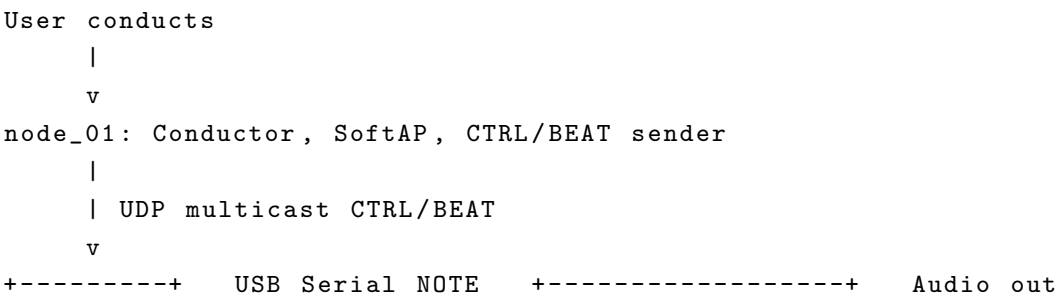
表 3: Processing 音源サブシステムの FBS

ID	機能	役割	WBS
F6.1	Serial 接続管理	Processing 起動時に 1 つの USB Serial ポートを開く．切断時は再接続待ちに遷移し，ユーザーがポートを再選択できるようにする．	124
F6.2	NOTE フレーム同期・復号	Serial から届くバイト列を内部バッファへ蓄積し，magic=0x4F52 を探して 20B 固定長の NOTE パケットへ復号する．version/type/seq/partId を検査する．	124
F6.3	パート設定管理	Mac 4 台構成では各 Processing が自パート 1 つだけを鳴らす．統合デモや検証用として，1 台で 4 パートを受け取るモードも持つ．	125
F6.4	ボイス管理	NoteOn/NoteOff, durationMs 経過による自動停止，同一 partId 内の重複発音，同時発音数上限を管理する．	125
F6.5	音色生成	partId に応じて金管波形またはドラム波形を選択し，ADSR エンベロープを適用して Minim の AudioOutput へ送る．	117/126
F6.6	画面表示・操作 IF	接続状態，partId, seq, 受信間隔，発音中ボイス数，音量，ミュート状態を表示し，キー操作でポート再接続，ミュート，テスト発音を行う．	124/125
F6.7	ログ・検証	受信時刻，発音時刻，seq 欠落，破棄理由を CSV ログとして保存し，結合試験で遅延・取りこぼし・音色確認に使えるようにする．	127

2.2 ユーザー目線のシステム構成

本書の主構成は，塩澤設計に合わせて「楽器ノード 1 台につき Mac 1 台」である．各 Mac 上の Processing は，自分に USB 接続された Arduino UNO R4 WiFi から NOTE だけを受け，自パートの音を鳴らす．

Listing 1 Processing 音源サブシステムを含む全体構成



node_02	----->	Mac 1 Processing	----->	Brass 1
node_03	----->	Mac 2 Processing	----->	Brass 2
node_04	----->	Mac 3 Processing	----->	Brass 3
node_05	----->	Mac 4 Processing	----->	Drum
+-----+		+-----+		

2.3 パート対応

表 4 partId と Processing 音色の対応

ノード	partId	役割	Processing 音色
node_02	0x02	金管 1（主旋律）	BrassVoice
node_03	0x03	金管 2（輪唱 1, 4 拍遅れ）	BrassVoice
node_04	0x04	金管 3（輪唱 2, 8 拍遅れ）	BrassVoice
node_05	0x05	ドラム（リズム伴奏）	DrumVoice

Mac 4 台構成では、partId は接続確認用としても利用する。たとえば Mac 1 は期待 partId を 0x02 に設定し、0x03–0x05 を受信した場合は画面に警告を表示して発音しない。統合デモモードでは期待 partId を ALL とし、4 パートすべてを鳴らす。

2.4 性能指標

表 5: Processing 側 MOP

ID	指標	目標値
PMOP-1	NOTE 受信から発音予約までの処理遅延	5 ms 以下
PMOP-2	Serial フレーム同期復帰	破損バイト混入後、次の正常 magic から 1 パケット以内に復帰
PMOP-3	partId 誤接続検出	期待 partId と異なる NOTE を 100% 警告表示し、発音しない
PMOP-4	同時発音管理	金管は各パート最大 4 ボイス、ドラムは最大 8 ボイスまで破綻なく再生
PMOP-5	音量安全性	velocity=127 でも AudioOutput がクリップしないよう総振幅を 0.8 以下に正規化
PMOP-6	音色根拠	金管の倍音比率と ADSR 値を表として管理し、FFT 解析結果で更新可能

3 基本設計：操作インターフェースと状態遷移

3.1 ユーザー操作

Processing 側は，デモ担当者または演奏準備担当者が操作する．観客は直接 PC を操作せず，指揮者の動きにより間接的にテンポと強弱を変える．

操作	対象	システム挙動
Processing 起動	Mac 担当者	Serial ポート一覧を表示し，前回保存したポートがあれば自動接続を試みる．
キー 1-4	Mac 担当者	期待 partId を 0x02-0x05 に切り替える．本番では Mac ごとに固定する．
キー a	Mac 担当者	統合デモモードに入り，partId 0x02-0x05 をすべて受け付ける．
キー r	Mac 担当者	Serial を閉じ，ポート再接続状態へ戻る．
キー m	Mac 担当者	ミュートを切り替える．ミュート中も受信ログは残す．
キー t	Mac 担当者	選択中 partId のテスト音を鳴らし，スピーカ接続と音量を確認する．
キー l	Mac 担当者	CSV ログ保存を開始・停止する．
指揮動作	指揮者	Arduino 側がテンポ・velocity を変化させ，NOTE の durationMs と velocity に反映される．

3.2 画面表示

Processing 画面は，演奏中の障害を即座に見つけるための最小 UI とする．表示項目は以下である．

- 接続状態：Disconnected / PortSelect / Ready / Playing / Error
- 接続ポート名，期待 partId，実際に受信した partId
- 直近 seq，seq 欠落数，破棄パケット数
- 直近 NOTE の noteNumber，velocity，gate，durationMs
- 発音中ボイス数，ミュート状態，ログ保存状態
- 波形モニタ（既存の pc_app/processing_test の表示を流用）

3.3 状態遷移

Listing 2 Processing 音源サブシステムの状態遷移

Boot	
-> PortSelect	: setup() completed, serial ports listed
PortSelect	
-> Ready	: selected port opened
-> Error	: port open failed
Ready	
-> Playing	: first valid NOTE received

```
-> PortSelect      : user presses r
Playing
-> Muted           : user presses m
-> Error           : serial exception or timeout > 3000 ms
-> Ready           : no NOTE for 1000 ms, connection still alive
Muted
-> Playing         : user presses m
-> Error           : serial exception
Error
-> PortSelect      : user presses r or auto retry interval elapsed
```

状態	意味	主な処理
Boot	Processing 起動直後	Minim, AudioOutput, 設定値, ログ ファイルを初期化する.
PortSelect	Serial ポート選択待ち	利用可能ポートを画面表示し, 設定済 みポートへ接続を試みる.
Ready	接続済み・NOTE 待ち	Serial バッファを空にし, magic 探索 を開始する.
Playing	正常受信・発音中	NOTE 復号, partId 検査, Voice 生成, 画面更新, ログ出力を行う.
Muted	受信継続・発音停止	NOTE は復号しログに残すが, Voice は生成しない.
Error	接続異常・復号不能	Serial を閉じ, 発音中 Voice を停止し, 破棄理由を表示する.

4 詳細設計：ハードウェア設計

Processing 担当範囲の主対象はソフトウェアであるが、ループリックのハードウェア設計項目に対応するため、PC 側に接続される構成要素・接続関係・動作を明確化する。

4.1 構成要素

区分	要素	数量	役割
H1	Arduino UNO R4 WiFi	4	node_02-05. 楽譜進行後, NOTE パケットを USB Serial で送信する.
H2	Mac または PC	4	各楽器ノード専属の Processing 実行機. 1 台につき 1Serial ポートだけ開く.
H3	USB Type-C ケーブル	4	各 Arduino と各 Mac を 1 対 1 で接続し, 給電と Serial 通信を兼ねる.
H4	スピーカまたは内蔵スピーカ	4	Processing の AudioOutput を再生する. 本番では各 Mac の音量を同程度に調整する.
H5	指揮者ノード	1	node_01. Processing には直接接続しないが, BEAT/CTRL により NOTE 発生時刻を決める.

4.2 接続設計

Listing 3 Mac 4 台構成の接続図

node_02	USB-C	----	Mac 1	----	speaker 1	:	partId 0x02, Brass 1
node_03	USB-C	----	Mac 2	----	speaker 2	:	partId 0x03, Brass 2
node_04	USB-C	----	Mac 3	----	speaker 3	:	partId 0x04, Brass 3
node_05	USB-C	----	Mac 4	----	speaker 4	:	partId 0x05, Drum

項目	設計
通信方式	USB CDC Serial. Arduino 側は Serial.write() で 20B 固定長バイナリを書き込む.
ボーレート	115200 bps. USB CDC では実効速度は十分だが, Arduino 側の NoteSenderConfig と統一する.
フレーミング	先頭 2B の magic=0x4F52 を同期マークとする. Serial 上のバイト列はリトルエンディアンなので 0x52 0x4F の順で現れる.
音声出力	Processing の Minim AudioOutput を各 Mac の既定出力へ送る. 会場では OS 音量を 70% 程度から開始し, クリップがないよう調整する.
誤接続対策	Mac ごとに期待 partId を固定し, 異なる partId を受信したら画面表示・ログ出力し発音しない.
故障時動作	USB が抜けた場合は Error へ遷移し, すべての発音を停止する. 再接続後, キー r で PortSelect へ戻す.

5 詳細設計：ソフトウェア設計

5.1 ファイル構成

実装時は、pc_app/orchestra_processing/に Processing スケッチを置く。Processing の仕様上、フォルダ名とメイン.pde 名は一致させる。

Listing 4 Processing 実装予定ファイル構成

```
pc_app/
'-- orchestra_processing/
  |-- orchestra_processing.pde    // setup(), draw(), keyPressed()
  |-- Config.pde                 // partId, serial port, audio params
  |-- OrcProtocol.pde            // packet structs, little-endian decoder
  |-- SerialFrameReader.pde      // byte buffer and magic sync
  |-- PartManager.pde            // part filter and voice allocation
  |-- Voice.pde                  // BrassVoice, DrumVoice, ADSR
  |-- ToneFactory.pde            // waveform tables
  |-- UIView.pde                 // status display and waveform monitor
  '-- Logger.pde                 // CSV logging
```

5.2 NOTE パケット定義

塩澤設計の OrcProtocol に合わせ、Processing 側は表 10 の 20B を読む。CTRL/BEAT は Processing へ直接来ないため、type=1 または type=2 を受信した場合は破棄する。

表 10: NOTE パケット

offset	field	type	Processing 側の扱い
0	magic	uint16	0x4F52 以外なら同期探索を続ける。
2	version	uint8	0x01 以外なら破棄し、version error としてログする。
3	type	uint8	3 のときのみ NOTE として処理する。
4	seq	uint32	前回 seq との差を取り、欠落・重複を検出する。
8	timestampMs	uint32	Arduino 側送信時刻。ログに保存し、遅延評価に使う。
12	partId	uint8	0x02–0x05. 期待 partId または ALL と一致する場合だけ発音する。
13	noteNumber	uint8	MIDI ノート番号。0 は休符または NoteOff 相当として扱う。
14	velocity	uint8	0–127. 発音振幅へ変換する。
15	gate	uint8	1=NoteOn, 0=NoteOff. その他の値は破棄する。
16	durationMs	uint16	NoteOn 時の自動 NoteOff 予定時刻を計算する。0 なら最小長 50 ms を使う。

offset	field	type	Processing 側の扱い
18	reserved	uint8[2]	0 以外でも発音は継続するが、ログに警告を残す。

5.3 クラス設計

クラス/構造体	主なフィールド・メソッド	役割
NotePacket	partId, noteNumber, velocity, gate, durationMs, seq, timestampMs	20B バイナリから復号した発音指令を表す。
SerialFrameReader	pushByte(), tryReadPacket(), dropUntilMagic()	Serial 入力を蓄積し、magic 同期と 20B 切り出しを行う。破損時は 1B ずつ捨てて再同期する。
PartConfig	expectedPartId, instrumentType, maxVoices	Mac ごとの担当パートと音色を定義する。
PartManager	handleNote(), noteOn(), noteOff(), update()	partId 検査、同時発音上限、durationMs 経過による停止を管理する。
Voice	start(), release(), isFinished()	1 つの発音単位。Minim の 0scil とエンベロープを保持する。
BrassVoice	makeBrassWaveform(), attackMs, releaseMs	金管 1-3 の音色を作る。倍音テーブルと ADSR 値は表で管理する。
DrumVoice	makeNoiseBurst(), decayMs	node_05 用。ノイズ成分と短い減衰でリズム伴奏を表現する。
UIView	drawStatus(), drawWaveform()	状態、受信内容、エラー、波形モニタを描画する。
Logger	logPacket(), logError(), flush()	結合試験用 CSV を保存する。

5.4 音色パラメータ

5.4.1 金管音色

金管音色は、加算合成による倍音テーブルと ADSR で作る。初版では Line による単純な直線減衰としていたが、金管らしさを出すため、本設計では最初から ADSR 相当のエンベロープを採用する。

表 12 金管音色の初期倍音比率

倍音	1	2	3	4	5	6	7
振幅	1.00	0.70	0.50	0.30	0.20	0.10	0.05

5.4.2 ドラム音色

塩澤案では node_05 がドラムであるため、4 パートすべてを金管で鳴らす初版設計は採用しない。ドラムは、ノイズ成分と短い減衰を用いる。最小実装では MIDI ノート番号ごとにキック、スネア、ハイハット風の 3 種類を割

表 13 金管 ADSR 初期値

項目	値	意味	調整方針
Attack	20 ms	息の入りの立ち上がり	遅すぎる場合は 10 ms へ短縮
Decay	40 ms	最大音量から安定音量へ移る時間	音が硬い場合に延長
Sustain	最大音量の 0.75 倍	音符中の持続音量	velocity に応じて変動
Release	60 ms	息を止めた後の余韻	音の切れが悪い場合は 40 ms へ短縮

り当てる.

noteNumber	音色	合成方針
36	Kick	低周波のサイン波を 50–80 Hz から下げ, Release 120 ms で減衰する.
38	Snare	ホワイトノイズ + 180 Hz 付近の短い共鳴, Release 90 ms.
42	Hi-hat	高域ノイズを短く鳴らし, Release 40 ms.
その他	Click	短いノイズで代替し, 未知の noteNumber でも無音にならないようにする.

5.5 FFT 解析の設計

WBS 127 は任意タスクだが, ADR-0007 の「倍音データを元に根拠を持った音色作り」と対応するため, 音色パラメータを外部データで更新可能な形にしておく.

- 1. フリー素材または録音した金管単音を `tools/`配下に置く.
- 2. Python で RMS 包絡線と FFT を計算し, 基音を 1.0 として第 2–第 7 倍音を正規化する.
- 3. 解析結果を CSV として保存する. 列は `harmonic`, `amplitude` とする.
- 4. Processing の `ToneFactory` は, 初期値をコード内に持ちつつ, CSV があれば読み込む.
- 5. 音色評価では, 初期値版と FFT 反映版を聞き比べ, 採用値を記録する.

6 詳細設計：処理フローの設計

6.1 起動処理

1. `setup()` で画面, Minim, AudioOutput, UI 状態, Logger を初期化する.
2. 設定ファイルまたはキー入力から期待 `partId` を決定する. Mac 1-4 では 0x02-0x05 に固定する.
3. `Serial.list()` でポート一覧を取得し, 前回ポートが存在すれば `Serial` を開く.
4. 接続成功なら Ready へ, 失敗なら PortSelect へ遷移する.
5. テスト音を鳴らし, スピーカと音量を確認できるようにする.

6.2 NOTE 受信フロー

Listing 5 Serial 受信から発音までの処理フロー

```
serialEvent()  
while serial has bytes:  
    reader.pushByte(serial.read())  
    while reader.tryReadPacket(packet):  
        if packet.version != 1:  
            log version error; continue  
        if packet.type != 3:  
            log non-NOTE packet; continue  
        if packet.gate is not 0 or 1:  
            log gate error; continue  
        if expectedPartId != ALL and packet.partId != expectedPartId:  
            log wrong part; show warning; continue  
        if packet.seq is duplicate:  
            log duplicate; continue  
        PartManager.handleNote(packet)
```

6.3 ボイス管理フロー

1. `gate=1` かつ `noteNumber>0` なら NoteOn として扱う.
2. `partId` から音色種別を決める. 0x02-0x04 は BrassVoice, 0x05 は DrumVoice である.
3. `velocity` を 0.0-1.0 へ正規化し, 総振幅上限 0.8 を超えないよう `maxAmp` を決める.
4. 同一 `partId` の発音数が上限を超える場合は, 最も古い Voice を Release へ移す.
5. `durationMs` から自動 NoteOff 時刻を計算する. 金管は最低 80 ms, ドラムは最低 30 ms とする.
6. `gate=0` または自動 NoteOff 時刻到達時に, 該当 `partId` の同一 `noteNumber` を Release へ移す.
7. Release 終了後, Voice を配列から削除し, Minim の接続を解除する.

6.4 例外処理

条件	処理	目的
magic 不一致	1B ずつ破棄して次の 0x52 0x4F を探す	バイトずれから復帰する.
version 不一致	パケット全体を破棄	旧仕様・別仕様との混在を防ぐ.

条件	処理	目的
type 不一致	パケット全体を破棄	CTRL/BEAT を誤って発音しない.
seq 重複	発音せずログのみ	二重発音を防ぐ.
seq 欠落	発音は継続し, 欠落数をログ	USB Serial ではまれだが検証可能にする.
partId 不一致	発音せず画面警告	Mac と Arduino の接続ミスを即発見する.
Serial 切断	全 Voice を Release し Error へ遷移	無限発音を防ぐ.
音量過大	全 Voice の振幅をスケールダウン	クリップと耳に痛い音を防ぐ.

6.5 主要コード断片

```

1 class NotePacket {
2     int version, type, partId, noteNumber, velocity, gate, durationMs;
3     long seq, timestampMs;
4 }
5
6 int u16le(byte[] b, int off) {
7     return (b[off] & 0xff) | ((b[off + 1] & 0xff) << 8);
8 }
9
10 long u32le(byte[] b, int off) {
11     return ((long)b[off] & 0xff)
12         | (((long)b[off + 1] & 0xff) << 8)
13         | (((long)b[off + 2] & 0xff) << 16)
14         | (((long)b[off + 3] & 0xff) << 24);
15 }
16
17 NotePacket decodeNote(byte[] frame) {
18     if (u16le(frame, 0) != 0x4F52) return null;
19     NotePacket p = new NotePacket();
20     p.version = frame[2] & 0xff;
21     p.type = frame[3] & 0xff;
22     p.seq = u32le(frame, 4);
23     p.timestampMs = u32le(frame, 8);
24     p.partId = frame[12] & 0xff;
25     p.noteNumber = frame[13] & 0xff;
26     p.velocity = frame[14] & 0xff;
27     p.gate = frame[15] & 0xff;
28     p.durationMs = u16le(frame, 16);
29     return p;
30 }

```

Listing 6 NOTE 復号の最小実装イメージ

```

1 Waveform makeBrassWaveform() {
2     return WavetableGenerator.gen10(
3         4096,
4         new float[] {

```

```
5         1.00f, 0.70f, 0.50f, 0.30f,  
6         0.20f, 0.10f, 0.05f  
7     }  
8 );  
9 }
```

Listing 7 金管波形生成の実装イメージ

7 検証計画

本書の仕様がループリック上の「実装に必要な設計」になっているかを確認するため、Processing 単体試験、Arduino との結合試験、音色評価を行う。

ID	試験項目	手順	合否基準
PT-1	NOTE 復号単体	20B の正常フレームを固定配列で与え、NotePacket へ復号する。	全フィールド一致
PT-2	フレーム同期復帰	先頭に破損バイトを混ぜた後、正常フレームを与える。	次の magic から復帰
PT-3	partId 誤接続	Mac 1 設定で partId 0x03 を入力する。	発音せず警告表示
PT-4	NoteOn/Off	gate=1 後, gate=0 または durationMs 経過を入力する。	無限発音しない
PT-5	同時発音上限	金管で 5 音以上を連続入力する。	古い Voice を Release し、上限 4 を維持
PT-6	ドラム音色	partId 0x05 で note 36/38/42 を入力する。	3 種類のリズム音として聞き分け可能
PT-7	受信遅延	受信時刻から Voice 生成までを millis() で 100 回記録する。	最大 5 ms 以下
PT-8	結合演奏	node_02-05 と Mac 4 台を接続し、カエルの歌等を演奏する。	4 パートが欠落なく鳴る

7.1 ループリック対応表

評価項目	目標評価	本書での対応
基本設計：機能一覧とシステム構成図	A	FBS 表, Mac 4 台構成図, partId 対応表, MOP を記載した。
基本設計：操作インターフェースと状態遷移	A	キー操作, 画面表示, Boot から Error までの状態遷移と状態定義を記載した。
詳細設計：ハードウェア設計	A	Arduino, Mac, USB, スピーカの数量・接続関係・動作・故障時挙動を記載した。
詳細設計：ソフトウェア設計	A	ファイル構成, NOTE パケット, クラス設計, 音色パラメータ, コード断片を記載した。
詳細設計：処理フローの設計	A	起動, 受信, 復号, 発音, 例外処理, ボイス停止, 検証フローを一貫して記載した。

8 今後の実装順序

1. pc_app/orchestra_processing/に最小スケッチを作り、Minim でテスト音を鳴らす。
2. SerialFrameReader と decodeNote() を実装し、固定バイト列で PT-1/2 を通す。

3. `PartManager` を実装し, `partId` 検査, `NoteOn/Off`, 自動停止を通す.
4. 金管 3 パートの `BrassVoice` を実装し, 既存の `HackInstrument` を `ADSR` 対応へ拡張する.
5. `node_05` 用の `DrumVoice` を実装する.
6. UI と CSV ログを実装し, Mac 4 台の接続ミスが見える状態にする.
7. Arduino 側の `NoteSenderModule` と結合し, PT-8 を実施する.

9 生成 AI の利用について

本改訂案の作成にあたり, 生成 AI を利用した. 利用範囲は, 既存資料のレビュー結果を踏まえた章立ての整理, ルーブリック項目との対応確認, および文章案の作成である. ただし, 採用する前提条件, NOTE パケット仕様, Mac 4 台構成, `partId` 対応, 試験項目については, リポジトリ内資料およびチーム内レビューを確認したうえで著者が判断する.